

Festival

Nayra este la un festival și joacă un joc al cărui mare premiu este o excursie la Laguna Colorada.

Jocul constă în utilizarea de jetoane pentru a cumpăra cupoane. Cumpărând un cupon se pot obține jetoane suplimentare. Scopul este să obțină cât mai multe cupoane posibil.

Ea începe jocul cu A jetoane. Există N cupoane, numerotate de la 0 la $N - 1$. Nayra trebuie să plătească $P[i]$ ($0 \leq i < N$) jetoane pentru a cumpăra cuponul i (și trebuie să aibă cel puțin $P[i]$ jetoane înainte de achiziție). Ea poate cumpăra fiecare cupon cel mult o dată.

Mai mult, fiecărui cupon i ($0 \leq i < N$) i se atribuie un **tip**, notat cu $T[i]$, care este un număr întreg **între 1 și 4, inclusiv**. După ce Nayra cumpără cuponul i , numărul rămas de jetoane pe care le are este înmulțit cu $T[i]$. Formal, dacă ea are X jetoane la un moment dat al jocului, și cumpără cuponul i (care necesită $X \geq P[i]$), atunci ea va avea $(X - P[i]) \cdot T[i]$ jetoane după cumpărare.

Sarcina ta este să stabilești ce cupoane ar trebui să cumpere Nayra și în ce ordine, pentru a maximiza numărul total de **cupoane** pe care le are la final. Dacă există mai multe secvențe de achiziții care realizează acest lucru, puteți prezenta oricare dintre ele.

Detalii de implementare

Trebuie să implementezi următoarea funcție.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
std::vector<int> T)
```

- A : numărul inițial de jetoane pe care le are Nayra.
- P : un tablou de lungime N care specifică prețurile cupoanelor.
- T : un tablou de lungime N care specifică tipurile de cupoane.

Această funcție este apelată exact o singură dată pentru fiecare caz de testare.

Funcția va returna un tablou R , care specifică achizițiile Nayrei după cum urmează:

- Lungimea lui R ar trebui să fie numărul maxim de cupoane pe care le poate cumpăra.
- Elementele tabloului sunt indicii cupoanelor pe care ar trebui să le cumpere, în ordine cronologică. Adică, ea cumpără mai întâi cuponul $R[0]$, apoi cuponul $R[1]$ și așa mai departe.
- Toate elementele lui R trebuie să fie unice.

Dacă nu se pot cumpăra cupoane, R ar trebui să fie un tablou vid.

Restricții

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ pentru fiecare i astfel încât $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ pentru fiecare i astfel încât $0 \leq i < N$.

Subtask-uri

Subtask	Scor	Restricții suplimentare
1	5	$T[i] = 1$ pentru fiecare i astfel încât $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ pentru fiecare i astfel încât $0 \leq i < N$.
3	12	$T[i] \leq 2$ pentru fiecare i astfel încât $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra poate cumpăra toate cupoanele N (într-o anumită ordine).
6	16	$(A - P[i]) \cdot T[i] < A$ pentru fiecare i astfel încât $0 \leq i < N$.
7	18	Fără restricții suplimentare.

Exemple

Exemplul 1

Considerăm următorul apel.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra are inițial $A = 13$ jetoane. Ea poate cumpăra 3 cupoane în ordinea de mai jos:

Cupon cumpărat	Preț cupon	Tip cupon	Număr de jetoane după cumpărare
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

În acest exemplu, Nayra nu poate cumpăra mai mult de 3 cupoane, iar secvența de achiziții descrisă mai sus este singura modalitate prin care ea poate cumpăra 3 dintre ele. Prin urmare, funcția va returna $[2, 3, 0]$.

Exemplul 2

Considerăm următorul apel.

```
max_coupons(9, [6, 5], [2, 3])
```

În acest exemplu, Nayra poate cumpăra ambele cupoane în orice ordine. Prin urmare, funcția ar trebui să returneze fie $[0, 1]$, fie $[1, 0]$.

Exemplul 3

Considerăm următorul apel.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

În acest exemplu, Nayra are un jeton, care nu este suficient pentru a cumpăra cupoane. Prin urmare, procedura ar trebui să returneze $[]$ (un tablou vid).

Grader-ul local

Format intrare:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Format ieșire:

```
S
R[0] R[1] ... R[S-1]
```

Aici, S este lungimea tabloului R returnat de `max_coupons`.